

WinniKnight: MindGuardians

Documentación de Integración Funcional — Android

Entrega 4 — Paso 4 | Fundamentos de Aplicaciones Web | Grupo 3

Integrante	Carné	Correo
Pablo Urbina	24001508	24001508@galileo.edu
Angel Camargo	24003664	angel.camargo@galileo.edu

1. Descripción del Proyecto

WinniKnight: MindGuardians gamifica el bienestar físico y mental. El usuario derrota monstruos que representan malos hábitos (sedentarismo, estrés, deshidratación) realizando acciones reales de salud. La app Android está construida con **Jetpack Compose** y conectada a **Firestore** y **Gemini AI** a través de **Cloud Functions** (Python).

La aplicación integra IA generativa de Gemini para tres propósitos diferenciados: narrar los ataques del héroe, narrar los contraataques del monstruo, y generar jefes dinámicos basados en las debilidades que el usuario reporta. Todo esto sucede en tiempo real durante el combate, creando una experiencia narrativa única y personalizada en cada sesión.

2. Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas claramente separadas:

- **App Android (Kotlin/Compose)** — interfaz de usuario y lógica de estado (ViewModel)
- **Firestore** — autenticación (Auth) y base de datos en tiempo real (Firestore)
- **Cloud Functions (Python)** — backend seguro que protege la API key de Gemini

La API key de Gemini nunca reside en el código fuente de la app Android. Vive en **Firestore Secrets** y solo es accesible desde el servidor de **Cloud Functions**, garantizando que no pueda ser extraída de la APK.

2.1 Flujo del Oráculo y Narración

Todos los endpoints de IA comparten el mismo flujo de comunicación:

Origen	Destino	Protocolo	Dato enviado
GameViewModel	GeminiRepository	Kotlin suspend fun	Prompt estructurado
GeminiRepository	Retrofit/OkHttp	HTTP POST	OracleRequest(message)
OkHttp	Cloud Function URL	HTTPS	JSON {message: ...}
Cloud Function	Gemini API	HTTPS	contents[parts[text]]
Gemini API	Cloud Function	HTTPS	candidates[0].text
Cloud Function	App Android	HTTPS	JSON {reply: ...}

2.2 Timeouts configurados (OkHttp)

- connectTimeout:** 30 segundos — tiempo máximo para establecer conexión con la Cloud Function.
- readTimeout:** 60 segundos — tiempo para recibir la respuesta de Gemini (generación de texto puede tardar).
- writeTimeout:** 30 segundos — tiempo para enviar el cuerpo de la solicitud.

3. Stack Tecnológico

Capa	Tecnología
UI	Jetpack Compose + Material3
Lenguaje	Kotlin 2.0.0
Estado	ViewModel + MutableState (Compose State Hoisting)

Autenticación	Firebase Auth (email/password)
Base de datos	Firebase Firestore (NoSQL en tiempo real)
Backend / IA	Firebase Cloud Functions (Python 3.13)
IA generativa	Gemini API (gemini-2.5-flash via Cloud Function)
HTTP (Android)	Retrofit 2.11.0 + OkHttp 4.12.0
Build system	Android Gradle Plugin 8.5.2
Min SDK	API 26 (Android 8.0 Oreo)
Tema visual	Dark space theme con colores neón personalizados

4. Dependencias

4.1 Android — gradle/libs.versions.toml

Librería	Versión	Uso
firebase-bom	33.1.0	Gestión centralizada de versiones Firebase
firebase-auth-ktx	vía BOM	Autenticación email/password
firebase-firestore-ktx	vía BOM	Base de datos NoSQL en tiempo real
retrofit	2.11.0	Cliente HTTP tipado para Cloud Function
converter-gson	2.11.0	Serialización/deserialización JSON automática
okhttp	4.12.0	Control de timeouts HTTP (connect/read/write)
google-services plugin	4.4.2	Conexión con proyecto Firebase
compose-bom	2024.08.00	Gestión de versiones Jetpack Compose
navigation-compose	2.7.7	Gestión de navegación (no usado en producción final)
lifecycle-runtime-ktx	2.8.3	Integración de ViewModel con Compose
activity-compose	1.9.1	setContent{} y soporte edge-to-edge

4.2 Cloud Function — functions/requirements.txt

Librería	Versión	Uso
firebase-functions	~0.5.0	Framework de Cloud Functions HTTP para Python
firebase-admin	~6.5.0	Acceso a servicios Firebase desde el servidor
requests	~2.31.0	Llamadas HTTP síncronas a la API de Gemini

5. Estructura de Firestore

La estructura es compartida entre Android y Web. Ambas plataformas leen y escriben los mismos documentos en el mismo proyecto Firebase (mindguardians-b07d3).

Colección / Documento	Campo	Tipo	Descripción
users/{uid}	displayName	String	Nombre único de héroe

users/{uid}	email	String	Correo del usuario
users/{uid}	heroLevel	Number	Nivel actual del héroe (1+)
users/{uid}	heroXp	Number	XP dentro del nivel actual (0-99)
users/{uid}	heroGold	Number	Oro acumulado disponible
users/{uid}	heroHp	Number	HP actual del héroe
users/{uid}	heroMaxHP	Number	HP máximo (base + bonus equipo)
users/{uid}	totalXp	Number	XP total histórico (ranking)
users/{uid}/battles/{id}	date	Timestamp	Fecha/hora del combate
users/{uid}/battles/{id}	habitType	String	Tipo de hábito del monstruo
users/{uid}/battles/{id}	result	String	"Victoria" o "Derrota"
users/{uid}/battles/{id}	goldEarned	Number	Oro ganado en el combate
users/{uid}/battles/{id}	xpEarned	Number	XP ganado en el combate
users/{uid}/inventory/{id}	id	String	ID del ítem del catálogo
users/{uid}/inventory/{id}	name	String	Nombre del ítem
users/{uid}/inventory/{id}	stat	String	Descripción del bonus
users/{uid}/inventory/{id}	emoji	String	Emoji del ítem
users/{uid}/inventory/{id}	price	Number	Precio pagado en oro
users/{uid}/inventory/{id}	bonusHp	Number	Bonus de HP máximo que otorga
users/{uid}/inventory/{id}	bonusPower	Number	Bonus de daño en % que otorga
users/{uid}/inventory/{id}	purchasedAt	Timestamp	Fecha de compra
shop_catalog/{id}	name	String	Nombre del ítem en tienda
shop_catalog/{id}	emoji	String	Emoji del ítem en tienda
shop_catalog/{id}	stat	String	Descripción del efecto
shop_catalog/{id}	price	Number	Precio en oro
shop_catalog/{id}	bonusHp	Number	HP máximo adicional
shop_catalog/{id}	bonusPower	Number	% adicional de daño

6. Estructura del Proyecto Android

Archivo / Carpeta	Descripción
MainActivity.kt	Entry point: MindGuardiansApp(), AppHeader, TabBar, GameScreen, AppFooter
GameState.kt (GameViewModel)	ViewModel principal: estado completo del juego, lógica de combate, compras, Oráculo
AuthViewModel.kt	Lógica de autenticación con StateFlow: login, register, logout, validaciones
data/FirebaseRepository.kt	Todas las operaciones Firestore: usuarios, batallas, inventario, tienda, ranking
data/GeminiRepository.kt	Cliente HTTP Retrofit: consultOracle, narrateHeroAttack, narrateMonsterAttack, validateDe
ui/screens/LoginScreen.kt	Pantalla de inicio de sesión con campo email y contraseña
ui/screens/RegisterScreen.kt	Pantalla de registro con nombre de héroe único, email y contraseña con confirmación
ui/screens/Screens.kt	DashboardScreen, ShopScreen, RankingScreen, OracleScreen, InventoryScreen y helpers
ui/screens/GuideScreen.kt	Guía interactiva del juego con secciones de hábitos y mecánicas
ui/components/ActionButtons.kt	AttackAction data class, attacksForMonster() (ataques dinámicos por tipo), ActionButtons, A
ui/components/MonsterCard.kt	Tarjeta del monstruo con animación flotante (tween), shake al recibir daño, barra HP con c
ui/components/HeroStats.kt	Panel de stats del héroe: barras HP y XP con gradiente, badges de nivel y oro
ui/components/BattleLog.kt	Log de combate con scroll automático, máximo 20 mensajes con colores por tipo
ui/components/VictoryModal.kt	Modal de victoria con animación spring, recompensas de oro y XP
ui/components/DefeatModal.kt	Modal de derrota con animación spring y botón de recuperación
ui/components/ReportModal.kt (ReportPanel)	Panel de reporte: campo de texto + botones Hazaña (bonus daño) y Debilidad (genera bos
ui/components/NavigationMenu.kt	Menú lateral overlay con navegación completa y botón de logout
ui/theme/Theme.kt	Tema oscuro espacial con colores neón: SpaceDark, CyanNeon, PurpleNeon, GoldNeon, C
functions/main.py	Cloud Function HTTP: valida request, llama a Gemini API con prompt RPG, retorna reply
functions/requirements.txt	Dependencias Python para la Cloud Function
gradle/libs.versions.toml	Version catalog centralizado de todas las dependencias del proyecto
app/src/main/AndroidManifest.xml	Configuración de la app: permisos de Internet, configuración de Activity

7. Flujos Funcionales Detallados

7.1 Flujo de Autenticación

1. **App inicia** → MainActivity verifica FirebaseAuth.getInstance().currentUser
2. **Sin sesión activa** → navega a LoginScreen
3. **Login exitoso** → isLoggedIn = true, key(sessionKey) recrea el GameViewModel limpio
4. **Sin cuenta** → navega a RegisterScreen
5. **Registro**: AuthViewModel.register() ejecuta:
 - a. Validación local de campos (nombre, email format, longitud contraseña, confirmación)
 - b. FirebaseRepository.isDisplayNameTaken() — consulta Firestore para unicidad del nombre
 - c. auth.createUserWithEmailAndPassword() — crea cuenta en Firebase Auth
 - d. updateProfile() — guarda displayName en el perfil de Firebase Auth
 - e. repository.createUserProfile() — crea documento en Firestore con stats iniciales

6. **Login automático** tras registro exitoso → isSuccess = true
7. **Logout** → FirebaseAuth.signOut(), isLoggedIn = false, sessionKey++ (destruye ViewModel)

7.2 Flujo de Carga de Datos del Héroe

1. GameViewModel.init() → loadUserThenInventory() (coroutine en viewModelScope)
2. FirebaseRepository.loadUserData() → lee users/{uid} de Firestore
3. Mapeo de campos: heroLevel, heroXp, heroGold, heroMaxHP, heroHp, totalXp, displayName
4. heroName: prioridad Firestore > Firebase Auth displayName > "Guerrero" (fallback)
5. isLoadingUser = false → UI puede mostrarse
6. repository.loadInventory() → IDs comprados (para filtrar tienda)
7. repository.loadFullInventory() → InventoryItemData completos con metadatos
8. applyEquipmentBonuses() → recalcula heroMaxHP con bonus del equipo
9. loadShopCatalog() + loadRanking() → en paralelo (coroutines separadas)

7.3 Flujo de Combate Completo

1. **Selección de ataque:** attacksForMonster() determina los 3 ataques disponibles según el tipo del monstruo actual
2. Usuario presiona botón → GameViewModel.attack(damage, actionName)
3. Cálculo de daño:
 - a. computeStatBonuses() suma bonusPower de todos los ítems del inventario
 - b. boostedDamage = damage + (damage × equipPower / 100)
 - c. Si bonusActive (por hazaña reportada): finalDamage = boostedDamage × 1.5
4. isAttacking = true → MonsterCard activa animación shake
5. GeminiRepository.narrateHeroAttack() → prompt enviado a Cloud Function para narración épica
6. addLog(narración, DAMAGE) → BattleLog actualizado (máx 20 entradas)
7. **Si monstruo derrotado:** isVictory = true → VictoryModal aparece
8. **Si monstruo vivo:** monsterCounterattack() (suspend fun):
 - a. Cálculo daño monstruo: maxHp/10 + bonus boss + (2..8).random()
 - b. GeminiRepository.narrateMonsterAttack() → narración del contraataque
 - c. heroHp reducido → si ≤ 0: isDefeat = true, saveBattle("Derrota", 0, 0)

7.4 Flujo de Victoria y Persistencia

1. VictoryModal muestra oro y XP a ganar (goldReward(), xpReward())
2. Usuario presiona «Siguiente Aventura» → GameViewModel.continueAfterVictory()
3. Actualización local: heroGold += gold, heroXp += xp, totalXp += xp
4. heroHp restaurado a heroMaxHP
5. **Level up:** si heroXp ≥ 100 → heroLevel++, heroXp -= 100, heroMaxHP += 10
6. monsterQueue: monstruo derrotado removido, si vacía se recarga MONSTERS[]
7. FirebaseRepository.saveBattle() → escribe en users/{uid}/battles/
8. FirebaseRepository.saveUserData() → actualiza users/{uid} con stats nuevos

7.5 Flujo del Panel de Reporte (Narrador del Oráculo)

El ReportPanel en CombatScreen permite dos acciones distintas con Gemini AI:

Hazaña (botón verde): El usuario describe una acción de salud realizada.

- GeminiRepository.validateDeed(deed) genera narración épica que celebra la hazaña
- bonusActive = true → próximo ataque hace ×1.5 daño (indicador visual mostrado)
- Debilidad** (botón rojo): El usuario reporta un mal hábito o debilidad.
- GeminiRepository.generateBoss(weakness) crea un jefe personalizado
- Respuesta parseada: NOMBRE: [...] / TIPO: [...] del reply de Gemini
- Monster boss (isBoss=true, 180 HP) insertado en posición 2 de monsterQueue

7.6 Flujo del Sistema de Inventario y Tienda

1. **Catálogo:** loadShopCatalog() lee shop_catalog/ de Firestore (fuente única de verdad)
2. ShopScreen filtra: items disponibles = catálogo – purchasedIds
3. Compra: GameViewModel.purchaseItem() valida oro suficiente y no duplicado
4. FirebaseRepository.spendGold() — transacción atómica en Firestore:
 - a. Lee heroGold actual, verifica suficiencia
 - b. userRef.update(heroGold - amount)
 - c. Agrega documento a users/{uid}/inventory/ con metadatos y Timestamp
5. applyEquipmentBonuses() recalcula heroMaxHP con todos los ítems comprados
6. InventoryScreen llama refreshFullInventory() al entrar para mostrar datos frescos

7.7 Flujo del Oráculo — Chat

1. OracleScreen inicializa con mensaje de bienvenida del Oráculo (en GameViewModel.init)
2. Usuario escribe hazaña → presiona «Consultar al Oráculo»
3. GameViewModel.consultOracle(message) → agrega mensaje user inmediatamente al chat
4. isOracleLoading = true → CircularProgressIndicator visible
5. GeminiRepository.consultOracle() → POST a Cloud Function con el mensaje
6. Cloud Function construye prompt RPG completo + llama Gemini API (gemini-2.5-flash)
7. Respuesta épica (máx 3 oraciones en español) → agregada al chat como rol «oracle»
8. isOracleLoading = false, LazyColumn hace auto-scroll al último mensaje

8. Cloud Function — Detalle de Implementación

La Cloud Function actúa como proxy seguro entre la app Android y la API de Gemini. Es el único punto donde la API key de Gemini existe en el sistema.

8.1 Endpoint

URL: <https://consult-oracle-ajesprbufa-uc.a.run.app/>

Método: **POST**

Content-Type: **application/json**

8.2 Comportamiento del endpoint

Escenario	Respuesta
Método OPTIONS	204 No Content con headers CORS (preflight)

Método != POST	405 Method Not Allowed
Body sin campo message	400 Bad Request
Error Gemini API	502 Bad Gateway con texto del error
Éxito	200 OK con JSON {reply: string}

8.3 Modelo Gemini y Prompt del Oráculo Chat

El endpoint principal usa el modelo **gemini-2.5-flash** (optimizado para velocidad y costo). Para el chat del Oráculo, el prompt establece al modelo como consejero de salud épico del juego. Para la narración de combate, el GameViewModel construye prompts especializados que se envían al mismo endpoint.

8.4 Prompts especializados por función (GeminiRepository.kt)

Función	Prompt enviado al endpoint	Máx palabras
consultOracle	Prompt RPG de bienestar definido en main.py	~3 oraciones
narrateHeroAttack	Narrador épico: ataque del héroe, daño, bonus	20 palabras
narrateMonsterAttack	Narrador épico: contraataque del monstruo	20 palabras
validateDeed	Celebrar hazaña + mencionar bonus daño	20 palabras
generateBoss	Crear jefe basado en debilidad (NOMBRE/TIPO)	Formato fijo

9. Componentes de UI — Detalles de Implementación

9.1 GameViewModel — Estado Global

El GameViewModel es el corazón de la aplicación. Gestiona toda la lógica de negocio:

Estado (MutableState)	Tipo	Descripción
heroHp / heroMaxHP	Int	HP actual y máximo del héroe
heroLevel / heroXp	Int	Nivel y experiencia (0-99 en nivel)
heroGold / totalXp	Int	Oro disponible y XP histórico total
heroName	String	Nombre del héroe (Firestore > Auth > fallback)
monsterHp	Int	HP actual del monstruo en combate
monsterQueue	mutableStateListOf	Cola de monstruos (3 base + bosses dinámicos)
bonusActive	Boolean	True si hay bonus ×1.5 por hazaña reportada
isVictory / isDefeat	Boolean	Control de modales de resultado
isBusy / isAttacking	Boolean	Locks durante animaciones y requests async
oracleMessages	mutableStateListOf	Historial del chat del Oráculo (role, text)
purchasedIds	mutableStateListOf	IDs de ítems comprados (filtra tienda)
inventoryItems	mutableStateListOf	Inventario completo con metadatos
shopCatalog	mutableStateListOf	Catálogo de tienda desde Firestore
ranking	mutableStateListOf	Top 10 héroes por totalXp

reportText	String	Campo de texto del panel de reporte
------------	--------	-------------------------------------

9.2 MonsterCard — Animaciones

MonsterCard implementa dos animaciones simultáneas con Compose:

Flotación: rememberInfiniteTransition con tween(1800ms, EaseInOutSine) en eje Y (0 a -10dp). Crea efecto de monstruo suspendido en el espacio.

Shake: Animatable activado con LaunchedEffect(isAttacking). keyframes de 400ms: -14f, +14f, -8f, 0f para simular impacto.

Barra HP: Color dinámico según porcentaje — CyanNeon (>60%), GoldNeon (30-60%), Red (<30%).

9.3 Sistema de Ataques Dinámicos — attacksForMonster()

La función attacksForMonster() analiza el tipo del monstruo y devuelve 3 ataques optimizados. Esto crea una capa de estrategia: el usuario debe adaptar sus acciones de salud al tipo de enemigo:

Tipo de monstruo	Ataques disponibles	Daño base
Gravedad / Peso	Hidratación, Salto Galáctico, Aliento	20 / 25 / 15
Caos / Mental / Mente	Zen Cósmico, Sueño Estelar, Aliento	30 / 25 / 20
Vacío / Estelar / Energía	Hidratación, Zen Cósmico, Sueño Estelar	20 / 20 / 20
Oscura / Fuerza	Salto Galáctico, Zen Cósmico, Hidratación	25 / 25 / 15
Default (no reconocido)	Elixir Estelar, Salto Galáctico, Zen Cósmico	15 / 20 / 25

9.4 Pantallas Implementadas

Pantalla (Screen enum)	Componente Kotlin	Descripción
COMBAT	CombatScreen(vm)	HeroStats + MonsterCard + BattleLog + ActionButtons + ReportPanel
DASHBOARD	DashboardScreen	Estadísticas con gráfica de barras semanal, filtros, stat cards
SHOP	ShopScreen(vm)	Tienda con catálogo de Firestore, grid 2 columnas, sección «Equipado»
RANKING	RankingScreen(vm)	Podio top 3 con alturas variables, lista scrollable posición 4-10
ORACLE	OracleScreen(vm)	Chat con Gemini AI, LazyColumn con auto-scroll, estados carga/error
INVENTORY	InventoryScreen(vm)	Lista de ítems comprados con emoji, nombre, stat, fecha de compra
GUIDE	GuideScreen()	Guía interactiva de mecánicas, hábitos y sistema de combate

10. Sistema de Progresión del Héroe

10.1 Mecánica de XP y Nivel

Cada monstruo derrotado otorga **XP** variable según su posición en la cola:

XP base: $30 + (\text{monsterQueue.size} \% 3) \times 15$ puntos.

XP boss: 80 puntos fijos (jefes generados por debilidades reportadas).

Al acumular **100 XP**: heroLevel++, heroXp -= 100, heroMaxHP += 10 (más vida permanente).

El campo **totalXp** nunca se resetea — es el valor usado para el ranking global.

10.2 Mecánica de Oro

Oro base por victoria: $20 + (\text{monsterQueue.size} \% 3) \times 10$ monedas.

Oro boss: 60 monedas fijas.

El oro se usa exclusivamente en la [Tienda](#) para comprar equipo permanente.

Cada ítem del catálogo tiene [bonusHp](#) y/o [bonusPower](#) que afectan el combate.

10.3 Sistema de Bonus por Hazaña

El ReportPanel introduce una mecánica de integración con hábitos reales de salud:

El usuario describe una acción real (ej: "Dormí 8 horas") → Gemini valida y narra la hazaña.

Se activa [bonusActive = true](#) → el panel de combate muestra «■ PODER x1.5 ACTIVO».

El próximo ataque aplica el multiplicador y luego [bonusActive = false](#) automáticamente.

10.4 Monstruos y Bosses Dinámicos

La monsterQueue implementa un sistema de roguelite con rotación infinita:

Monstruos base: [Titán del Sedentarismo](#) (100HP), [Cometa de la Fatiga](#) (120HP), [Nebulosa del Estrés](#) (150HP).

Al reportar una debilidad, Gemini genera un boss con nombre y tipo únicos ([isBoss=true](#), 180 HP).

El boss se inserta en posición 2 de la cola (se enfrenta después del monstruo actual).

Al vaciar la cola, se recarga MONSTERS[] completo para continuidad infinita.

11. Configuración e Instalación

11.1 Requisitos

- Android Studio Hedgehog (2023.1) o superior
- Android SDK 26+ (Android 8.0 Oreo mínimo)
- Cuenta de Firebase con proyecto MindGuardians configurado
- Firebase CLI instalado: `npm install -g firebase-tools`
- Python 3.13 para Cloud Functions

11.2 Pasos de Instalación

1. Clonar el repositorio del proyecto
2. Abrir la carpeta MindGuardians en Android Studio
3. Descargar google-services.json de Firebase Console y colocarlo en app/
4. Sincronizar Gradle (Build > Sync Project with Gradle Files)
5. Ejecutar en emulador (API 26+) o dispositivo físico conectado

11.3 Configurar API Key de Gemini (solo primera vez)

La API key NUNCA va en el código fuente. Se configura como Firebase Secret:

```
firebase functions:secrets:set GEMINI_API_KEY
```

6. Ingresar la API key cuando Firebase CLI la solicite

11.4 Deploy de Cloud Functions

```
firebase deploy --only functions
```

Verificar que la Cloud Function aparezca como activa en la consola de Firebase.

12. Seguridad

Aspecto de seguridad	Implementación
API key de Gemini	Firebase Secrets — nunca en código fuente ni en la APK
google-services.json	.gitignore — nunca subido al repositorio
Autenticación	Firebase Auth maneja sesiones, tokens JWT y renovación automática
Sesión y logout	signOut() + key(sessionKey++) destruye y recrea ViewModel limpio
Validación de compras	spendGold() verifica gold en Firestore antes de descontar (server-side)
Unicidad de nombre de héroe	isDisplayNameTaken() consulta Firestore antes de crear cuenta
Reglas de Firestore	Modo Test durante desarrollo — reglas de producción en entrega final
Errores de Firebase Auth	mapFirebaseError() traduce códigos técnicos a mensajes amigables
CORS en Cloud Function	Access-Control-Allow-Origin: * + manejo de preflight OPTIONS

13. Sincronización con la Plataforma Web

Ambas plataformas comparten el mismo proyecto Firebase (mindguardians-b07d3) y la misma colección users en Firestore:

- Un usuario registrado en Android puede iniciar sesión en la web con las mismas credenciales
- El progreso del héroe (level, gold, xp, hp) se refleja en ambas plataformas
- Las batallas guardadas desde Android aparecen en el historial web
- El ranking global es compartido — ordenado por totalXp en ambas plataformas
- El catálogo de la tienda (shop_catalog/) es leído por ambas plataformas
- Los ítems comprados en cualquier plataforma aparecen en el inventario de la otra

14. Tema Visual — Space Dark Theme

El tema visual está definido en ui/theme/Theme.kt con una paleta de colores neón sobre fondo espacial oscuro:

Token	Hex	Uso principal
SpaceDark	#0a0a1a	Fondo principal más oscuro
SpaceDeep	#0d0d2b	Fondo de cards y modales
SpaceMid	#141432	Fondo intermedio (gradientes)
SpaceBlue	#1a1a4a	Fondo de header/footer
CyanNeon	#06b6d4	Accent principal, tabs activos, HP bars
PurpleNeon	#7c3aed	Borders, badges, elementos secundarios
PurpleLight	#a78bfa	Textos secundarios púrpura

GoldNeon	#f59e0b	Oro, recompensas, logo animado
GoldDark	#d97706	Borders del oro
GreenNeon	#22c55e	Oráculo, hazañas, éxito
BorderPurple	#3b1d8a	Bordes de cards
TextWhite	#e2e8f0	Texto principal
TextMuted	#94a3b8	Texto secundario/placeholder